Fundamentals of theory of computation I.

Lecture 1

Krisztián Tichler
ktichler@inf.elte.hu

Books

- A. Salomaa, Formal Languages, Academic Press, 1973.
- J. E. Hopcroft, Rajeev Motwani, J.D. Ullman, Introduction to Automata Theory, Languages, and Computation. Second Edition. Addison-Wesley, 2001.
- A. V. Aho, R. Sethi, J. D. Ullman: Compilers - Principles, Techniques, and Tools, 2nd ed. Pearson Education Ltd., 2014.
- A. Salomaa, G. Rozenberg (eds.): The Handbook of Formal Languages I., II., Springer Publishing Company, 1997
- M. Sipser, Introduction to the Theory of Computation, 2nd ed., Thomson, 2006.

Topics

- Basic concepts and notations. Words, languages.
- Grammars, Chomsky hierarchy.
- Operations on languages.
- Closure properties of Chomsky classes.
- Regular grammars, regular languages, regular expressions.
- Context free grammars and languages. Reduced grammar, normal forms, derivation tree.
- Context sensitive and phrase structure grammars. Essentially noncontracting grammars, normal forms.
- Finite automata. Properties, algorithms. Their relation to regular grammars.
- Pushdown automata. Their relation to context-free languages.
- Algorithmic properties of Chomsky language classes.
- Syntactic parsing. CYK algorithm. LL(k) grammars. Unambiguous grammars.

In which fields can formal languages, grammars, automata be used?

- Computer processing of natural languages. (Mathematical model.)
- Theory of programming languages. (Mathematical model.)
- Theory of compilers.
- Coding theory.
- Picture processing.
- String matching.
- Modelling biological progressions (e.g., Lindenmayer systems).
- Bioinspired models of computation.

Basic concepts and notations

alphabet, words, length of a word

An **alphabet** V is a finite, nonempty set of symbols. The elements of an alphabet are called **letters**.

A finite sequence (or string) of letters of V is called a **word over** V.

For $u \in V^*$, the **length** of u is the number of letters of u and it is denoted by $|u|$. So if $u = t_1 \cdots t_n$, then $|u| = n$.

The string of length zero is denoted by ε. So $|\varepsilon| = 0$.

Example:
Let $V = \{a, b\}$, then the 2 letters of V are a and b. $abba$ and $aabba$ are words over V. $|abba| = 4$ and $|aabba| = 5$.

$aabba$ and $baaba$ are different words even though both of them contain three a's and two b's. Order of letters matters.

Basic concepts and notations

set of all words

The set of all words over V (including the empty word, ε) is denoted by V^*.

$V^+ = V^* \setminus \{\varepsilon\}$ is **the set of all nonempty words over** V.

Example: Let $V = \{a, b\}$, then
$V^* = \{\varepsilon, a, b, aa, ab, ba, bb, aaa, aab, \ldots\}$ and
$V^+ = \{a, b, aa, ab, ba, bb, aaa, aab, \ldots\}$.

The words of V^* is listed in a **length-lexicographic (shortlex) order**. In general, shortlex order is defined as follows.

- there is a total order given on the letters (a is smaller than b in the previous example),
- shorter words are before the longer ones,
- 2 words of the same length are ordered lexicographically (i.e., as in the lexicon, scanning them from left to right, the first different letter determines their order).

String operations

concatenation

Let V be an alphabet and let $u = s_1 \cdots s_n$ and $v = t_1 \cdots t_k$ be words over V ($s_1, \ldots, s_n, t_1, \ldots, t_k \in V$). Then $uv := s_1 \cdots s_n t_1 \cdots t_k$ is called the **concatenation** of u and v. (The sequence of letters of u is followed by the sequence of letters of v.)

So we have $|uv| = |u| + |v|$.

Example:
Let $V = \{a, b\}$. Furthermore, let $u = abb$ and $v = aaaba$ be words over V. Then we have $uv = abbaaaba$ and $vu = aaabaabb$.

String operations

concatenation

Concatenation is an associative but **non-commutative** operation.

- In most of the cases uv is different from vu ($u, v \in V^*$) (Remark: Concatenation is commutative for unary alphabets, but it is a very special case.)
- $u(vw) = (uv)w$ holds for any $u, v, w \in V^*$ (associativity).

V^* is closed under the concatenation operation. I.e., $uv \in V^*$ holds for any $u, v \in V^*$.

ε is a unit element for concatenation (i.e., $u = u\varepsilon = \varepsilon u$ holds for any $u \in V^*$).

So, V^* along with the concatenation operation is a semi-group with a unit element ε.

String operations

*i*th power

Let i be a nonnegative integer and let u be a word over V ($u \in V^*$). The ***i*th power** of u is the i-fold conatenation of u by itself and denoted by u^i.

Formally, $u^0 := \varepsilon$, $u^i := uu^{i-1} (i \geq 1)$.

Then $u^{n+k} = u^n u^k$ holds for $k, n \in \mathbb{N}$
($\mathbb{N}$ is a notation for the set of natural numbers: 0,1,2 ...)

Example:
Let $V = \{a, b\}$ and $u = abb$. Then $u^0 = \varepsilon$, $u^1 = abb$, $u^2 = abbabb$ and $u^3 = abbabbabb$.

Attention! $(ab)^3 \neq a^3b^3$

$(ab)^3 = ababab$, $\quad a^3b^3 = aaabbb$.

String operations

Reversal

Let u be a word over V. The **reversal** (or mirror image) of u, denoted by u^R, is the sequence of letters of u, but in a reversed order.
Formally, if $u = a_1 \cdots a_n$, $a_i \in V$, $1 \leq i \leq n$, then $u^R = a_n \cdots a_1$, i.e., the ith letter of u^R is the same as the $(|u| + 1 - i)$th letter of u $(1 \leq i \leq |u|)$.

Example:
Let $V = \{a, b\}$. Furthermore, let $u = abba$ and $v = baabba$ be words over V. Then $u^R = abba$ and $v^R = abbaab$.

In the case, when $u = u^R$ holds, u is called a **palindrome**.

So, *abba* is a palindrome, while *baabba* is not.

The following properties can be easily checked.

- $\varepsilon^R = \varepsilon$
- $(u^R)^R = u$
- $(uv)^R = v^R u^R$
- $(u^i)^R = (u^R)^i$, holds for all $i \in \mathbb{N}$.

Basic concepts and notations

subword, prefix, suffix

Let V be an alphabet and u, v be words over V. u is called a **subword** (or substring) of v if $v = xuy$ holds for some $x, y \in V^*$. u is a **proper subword** of v if both $u \neq v$ and $u \neq \varepsilon$ hold.

If $x = \varepsilon$ holds in the previous formula, then u is called a **prefix** of v. In the case of $y = \varepsilon$ u is called a **suffix** of v.

A prefix/suffix u of v is called a **proper prefix/suffix** if $u \neq \varepsilon$ and $u \neq v$.

Basic concepts and notations

subword, prefix, suffix

Example:

Let $V = \{a, b\}$ and $u = abba$.

subwords of u: $\varepsilon, a, b, ab, bb, ba, abb, bba, abba$.

proper subwords of u: $a, b, ab, bb, ba, abb, bba$.

prefixes of u: $\varepsilon, a, ab, abb, abba$.

suffixes of u: $\varepsilon, a, ba, bba, abba$.

proper prefixes of u: a, ab, abb.

proper suffixes of u: a, ba, bba.

Attention! *abb* is not a suffix of u. In general, suffixes are not prefixes of the reversed word.

Basic concepts and notations

languages

Let V be an alphabet. A subset L of V^* is called a **language** over V.

The empty language (the language containing no words) is denoted by $\emptyset$.

A language is finite if it consists of a finite number of words (elements), otherwise it is infinite.

Examples:

Let $V = \{a, b\}$ be an alphabet.

- $L_1 = \{a, ba, ababb, \varepsilon\}$.
- $L_2 = \{a^i b^i \mid i \geq 0\}$.
- $L_3 = \{uu^R \mid u \in V^*\}$.
- $L_4 = \{a^n \mid n \geq 1\}$.
- $L_5 = \{u \mid u \in \{a, b\}^+, u \text{ contains as many } a\text{'s than } b\text{'s}\}$.

L_1 is a finite language, the others are infinite.

Operations on languages

set operations

Let V be an alphabet and let L_1, L_2 be languages over V (i.e., $L_1 \subseteq V^*$ and $L_2 \subseteq V^*$).

$L_1 \cup L_2 := \{u \mid u \in L_1 \text{ or } u \in L_2\}$ (**union** of L_1 and L_2)

$L_1 \cap L_2 := \{u \mid u \in L_1 \text{ and } u \in L_2\}$ (**intersection** of L_1 and L_2)

$L_1 - L_2 := \{u \mid u \in L_1 \text{ and } u \notin L_2\}$ (**difference** of L_1 and L_2)

Examples:

$L_1 = \{\varepsilon, ab, abb\}, L_2 = \{ab, bab\}$. Then
$L_1 \cup L_2 = \{\varepsilon, ab, abb, bab\}$,
$L_1 \cap L_2 = \{ab\}$,
$L_1 - L_2 = \{\varepsilon, abb\}$.

The **complementary language** of $L \subseteq V^*$ is the language $\bar{L} = V^* - L$.

Example:

$V = \{a\}$, $L = \{a^{2n+1} \mid n \in \mathbb{N}\}$. Then $\bar{L} = \{a^{2n} \mid n \in \mathbb{N}\}$

Operations on languages

concatenation

Let V be an alphabet and $L_1, L_2 \subseteq V^*$. The **concatenation** of L_1 and L_2 is the language $L_1 L_2 = \{u_1 u_2 \mid u_1 \in L_1, u_2 \in L_2\}$.

Concatenation is an associative, but **non-commutative** operation.

Examples:

$V = \{a, b\}$, $L_1 = \{ab, bb\}$, $L_2 = \{\varepsilon, a, bba\}$,

$L_1 L_2 = \{ab\varepsilon, bb\varepsilon, aba, bba, abbba, bbbba\}$
$= \{ab, bb, aba, bba, abbba, bbbba\}$. (all pairs!)

$L_3 = \{a, ab\}$, $L_4 = \{bc, c\}$

$L_3 L_4 = \{abc, ac, abbc\}$, $|L_3 L_4| < 2 \cdot 2 = 4$, since abc can be produced in 2 ways.

$L_5 = \{a^{2n} b^{2n} \mid n \in \mathbb{N}\}, L_6 = \{a^{3n} b^{3n} \mid n \in \mathbb{N}\}$

$L_5 L_6 = \{a^{2n} b^{2n} a^{3k} b^{3k} \mid n, k \in \mathbb{N}\}$.

$L_5 L_6$ contains all pairs, e.g., concatenation of 46th element of L_5 and 87th element of L_6. $(n = 46, k = 87)$

Operations on languages

ith power

Observations: $\{\varepsilon\}L = L\{\varepsilon\} = L$ holds for all languages L.

$L_1(L_2 L_3) = (L_1 L_2)L_3$ (associativity)

$\{\varepsilon\}$ is a unit element of language concatenation since $L = L\{\varepsilon\} = \{\varepsilon\}L$ holds for any $L \subseteq V^*$.

So, the powerset of V^* along with the language concatenation operation forms a semi-group. It has a unit element $\{\varepsilon\}$.

The **ith power** of a language L $(i \in \mathbb{N})$ is defined as follows.
$L^0 := \{\varepsilon\}$, $L^i := LL^{i-1}$ $(i \geq 1)$.

I.e., L^i is the i-fold concatenation of L by itself.
We have $L^{k+n} = L^k L^n$ $(k, n \in \mathbb{N})$.

Example: $L = \{a, bb\}$.

$L^0 = \{\varepsilon\}$,
$L^1 = \{a, bb\}$,
$L^2 = \{a\,a, a\,bb, bb\,a, bb\,bb\}$,
$L^3 = \{a\,a\,a, a\,a\,bb, a\,bb\,a, a\,bb\,bb, bb\,a\,a, bb\,a\,bb, bb\,bb\,a, bb\,bb\,bb\}$.

Operations on languages

iterative closure (Kleene star)

$L^* = \bigcup_{i \geq 0} L^i$ is called the **iterative closure** of L. (Other names: Kleene closure, Kleene star).

$L^+ = \bigcup_{i \geq 1} L^i$ is called the **positive iterative closure** (or Kleene plus) of L.

Observation:
We have $L^+ = L^*$, if $\varepsilon \in L$ and $L^+ = L^* - \{\varepsilon\}$ if $\varepsilon \notin L$.

Example: $L = \{a, bb\}$.

$L^0 = \{\varepsilon\}$,
$L^1 = \{a, bb\}$,
$L^2 = \{a\,a, a\,bb, bb\,a, bb\,bb\}$,
$L^3 = \{a\,a\,a, a\,a\,bb, a\,bb\,a, a\,bb\,bb, bb\,a\,a, bb\,a\,bb, bb\,bb\,a, bb\,bb\,bb\}$,
$L^4 = \{a\,a\,a\,a, a\,a\,a\,bb, a\,a\,bb\,a, \ldots\}$, ...

L^* contains all the above words. We have $L^+ = L^* - \{\varepsilon\}$ in this example.

Operations on languages

reversed language

Let V be an alphabet and $L \subseteq V^*$. Then $L^R := \{u^R \mid u \in L\}$ is called the **reversal** (or mirror image) of the language L.

Example:
Let $V = \{a, b\}$, $L = \{ba^{2n+1} \mid n \in \mathbb{N}\}$, then $L^R = \{a^{2n+1}b \mid n \in \mathbb{N}\}$.

Some properties of language operations. (It's an exercise to check them.)

- $(L^R)^R = L$
- $(L_1 L_2 \cdots L_n)^R = L_n^R \cdots L_2^R L_1^R$
- $(L^i)^R = (L^R)^i$ holds for all $i \geq 0$,
- $(L^*)^R = (L^R)^*$,
- $L^* L^* = L^*$,
- $(L^*)^* = L^*$,
- $(L_1 \cup L_2)^* = (L_1^* L_2^*)^*$,
- $L_1(L_2 \cup L_3) = L_1 L_2 \cup L_1 L_3$, but $L_1(L_2 \cap L_3) \neq L_1 L_2 \cap L_1 L_3$.

Operations on languages

prefix closure, suffix closure

Let V be an alphabet and $L \subseteq V^*$.

$\mathrm{Pre}(L) = \{u \mid u \in V^*, uv \in L \text{ holds for some } v \in V^*\}$ is called the **prefix closure** of L.

$\mathrm{Suf}(L) = \{u \mid u \in V^*, vu \in L \text{ holds for some } v \in V^*\}$ is called the **suffix closure** of L.

Observations: $L \subseteq \mathrm{Pre}(L)$, $L \subseteq \mathrm{Suf}(L)$,

$\mathrm{Pre}(\mathrm{Pre}(L)) = \mathrm{Pre}(L)$, $\mathrm{Suf}(\mathrm{Suf}(L)) = \mathrm{Suf}(L)$.

Example:
$L = \{a^{2n+1}b \mid n \in \mathbb{N}\}$.

$\mathrm{Pre}(L) = \{a^{2n+1}b \mid n \in \mathbb{N}\} \cup \{a^n \mid n \in \mathbb{N}\}$.

$\mathrm{Suf}(L) = \{a^n b \mid n \in \mathbb{N}\} \cup \{\varepsilon\}$.

String homomorphism

Let V_1 and V_2 be alphabets. A mapping $h : V_1^* \to V_2^*$ is called a (string) **homomorphism** if

- for all $u \in V_1^*$ there exists exactly one $v \in V_2^*$, such that $h(u) = v$ holds and
- $h(uv) = h(u)h(v)$ holds for all $u, v \in V_1^*$.

Observations:

- Since $h(\varepsilon) = h(\varepsilon\varepsilon) = h(\varepsilon)h(\varepsilon)$ we have $|h(\varepsilon)| = 0$ implying $h(\varepsilon) = \varepsilon$.
- For all $u = a_1 a_2 \cdots a_n$, where $a_i \in V_1$, $1 \leq i \leq n$ holds we have $h(u) = h(a_1)h(a_2)\cdots h(a_n)$. This means, that h is defined by the images of the letters. Images of the letters determine h for all $u \in V_1^*$.

Operations on languages

homomorphic image

Let $h : V_1^* \to V_2^*$ be a homomorphism. h is called **ε-free**, if $h(u) \neq \varepsilon$ holds for all $u \in V_1^+$.

Let $h : V_1^* \to V_2^*$ be a homomorphism and $L \subseteq V_1^*$.

$h(L) = \{h(u) \mid u \in L\} \subseteq V_2^*$ is called the **homomorphic image** of L by h.

Example:
$V_1 = \{a, b\}, V_2 = \{b, c\}, L = \{a^n b a^n \mid n \in \mathbb{N}\}$
$h(a) = cc, h(b) = cbb$.

Then $h(L) = \{c^{2n+1} b^2 c^{2n} \mid n \in \mathbb{N}\}$.

Ways of defining formal languages

- Listing the elements.

E.g., $L_1 = \{ab, ba, abba, ca\}$

- By a formula.

E.g., $L_2 = \{a^n b^n \mid n \in \mathbb{N}\}$

- By giving a regular expression describing the language.

E.g., $L_3 = \{a^{2n+1} b \mid n \in \mathbb{N}\} \cup \{a^n \mid n \in \mathbb{N}\}$.

A regular expression describing L_3 is $(aa)^*ab + a^*$. (Will be discussed on a later lecture.)

- Giving an algorithm producing exactly the words of the language on its output.

$L_4 = \{0, 1\}^*$ is an infinite language. The shortlex order of $\{0, 1\}^*$ can be considered as an algorithm listing exactly the elements of L_4.

Ways of defining formal languages

- By (generative) grammars.

Grammars are **synthetising** tools. Starting from a single symbol words can be built. At each step a substring of the current string is replaced by another string according to a production rule. Those words that can be produced form a language. I.e., the set of production rules determines a language.

- By mathematical machines (automata).

Automata are **analysing** tools. In most of the cases their input is a single word and their output is a bit. According to the rules of the automata inputs are processed, producing an answer 'yes' for certain inputs. The set of these words forms a language, determined by the set of rules of the machine.

Attention! Not all languages can be described by all of these tools. E.g., we shall see, that not all languages generated by a grammar can be described by a regular expression. Even grammars are not strong enough to describe all the languages over $\{0, 1\}$.

Families (or classes) of languages

Let X be a set. $\mathcal{P}(X)$ denotes the powerset of X, i.e., $\mathcal{P}(X) = \{A \mid A \subseteq X\}$.

A set of languages over a given alphabet is called a **family of languages** (or class of languages).

if V is an alphabet, then

- $a \in V$ is a letter,
- $u \in V^*$ is a word,
- $L \subseteq V^*$ or $L \in \mathcal{P}(V^*)$ is a language,
- $\mathcal{L} \subseteq P(V^*)$ or $\mathcal{L} \in \mathcal{P}(\mathcal{P}(V^*))$ is a family of languages.

Examples: $V = \{a, b\}$

- the family of finite languages over V,
- the family of such languages over V, that contains only words starting with a b,
- the family of languages over V, that can be described by a regular expression.

Grammars

Definition

A (generative) **grammar** is a quadruple $G = (N, \Sigma, P, S)$, where

- N and Σ are disjoint alphabets (i.e., $N \cap \Sigma = \emptyset$). N is called the alphabet of **nonterminal** symbols, while Σ is the alphabet of **terminal** symbols.
- $S \in N$ is called the **start symbol** (or initial symbol),
- P is a set of tuples (x, y), where $x, y \in (N \cup \Sigma)^*$ and x contains at least one nonterminal symbol.

The elements of P are called (production) **rules** (or rewriting rules).

Instead of a tuple (x, y) we use the notation $x \to y$ in most of the cases.

We can do it if '$\to$' is not an element of $N \cup \Sigma$. From now on, we assume, that this property holds and we shall use $\to$'s in rules. x will be referred as the left hand side, y will be referred as the right hand side of the rule.

Grammars

Examples:

$G_1 = (\{S, A, B\}, \{a, b, c\}, \{S \to c, S \to AB, A \to aA, B \to \varepsilon, abb \to aSb\}, S)$ is **not a grammar**, since all the left hand sides of the rules should contain at least one nonterminal. Rule $abb \to aSb$ does not have any nonterminal on its left hand side.

$G_2 = (\{S, A, B, C\}, \{a, b, c\}, \{S \to a, S \to AB, A \to Ab, B \to \varepsilon, aCA \to aSc\}, S)$ is a grammar.

Grammars

one step derivation

Definition

Let $G = (N, \Sigma, P, S)$ be a grammar and $u, v \in (N \cup \Sigma)^*$. v can be **derived** form u **in one step** in G, denoted by $u \Rightarrow_G v$, if $u = u_1 x u_2$ and $v = u_1 y u_2$ hold, where $u_1, u_2 \in (N \cup \Sigma)^*$ and $x \to y \in P$.

Remark: In several cases it is clear in which grammar G the derivation takes place. In these cases the subscript G can be omitted, i.e., $\Rightarrow$ can be written instead of $\Rightarrow_G$.

Example:

$G_2 = (\{S, A, B, C\}, \{a, b, c\}, \{S \to a, S \to AB, A \to Ab, B \to \varepsilon, aCA \to aSc\}, S)$.

Then $BB\mathbf{aCA}a \Rightarrow BB\mathbf{aSc}a$, $\mathbf{A}BB \Rightarrow \mathbf{Ab}BB$, $B\mathbf{B} \Rightarrow B$.

Grammars

multistep derivation

Definition

Let $G = (N, \Sigma, P, S)$ be a grammar and $u, v \in (N \cup \Sigma)^*$. v can be **derived** (in multiple steps) from u if either $u = v$ or there exist an integer $n \geq 1$ and words $w_0, \ldots w_n \in (N \cup \Sigma)^*$, such that $w_{i-1} \Rightarrow_G w_i$ $(1 \leq i \leq n)$, $w_0 = u$ and $w_n = v$. Notation: $u \Rightarrow_G^* v$.

Those words, that can be derived from the start symbol are called **sentential forms** of G.

Grammars

multistep derivation

Example:

$G_2 = (\{S, A, B, C\}, \{a, b, c\}, \{S \to a, S \to AB, A \to Ab, B \to \varepsilon, aCA \to aSc\}, S)$.

Then $BB\mathbf{aCA}a \Rightarrow BB\mathbf{aSc}a$ és $BBa\mathbf{Sc}a \Rightarrow BBa\mathbf{AB}ca$, so $BBaCAa \Rightarrow^* BBaABca$.

$S \Rightarrow AB \Rightarrow AbB \Rightarrow AbbB$, so $AbbB$ is a sentential form.

Remark: $\Rightarrow_G$ is a binary relation on the groundset $(N \cup \Sigma)^*$. Relation $\Rightarrow_G^*$ is actually the reflexive, transitive closure of relation $\Rightarrow_G$ (to be explained on next slides).

Reflexive, transitive closure

- Let $X_1, \ldots X_n$ be sets and $R \subseteq X_1 \times \cdots \times X_n$. Then R is called an **n-ary relation**. ($n = 2$: binary relation.)
- The **composition** of binary relations $R \subseteq X \times Y$ and $S \subseteq Y \times Z$ is the binary relation $R \circ S = \{(x, z) \subseteq X \times Z \mid \exists y \in Y : (x, y) \in R \wedge (y, z) \in S\}$.
- If $R \subseteq X \times X$, then $S := R \cup \{(x, x) \mid x \in X\}$ is called the **reflexive closure** of R,
- If $R \subseteq X \times X$, then recursion $R^1 := R$, $R^i := R \circ R^{i-1}$ $(i \geq 2)$ defines the **powers** of R,
- If $R \subseteq X \times X$, then $R^+ := \bigcup_{\{i \mid i \geq 1\}} R^i$ is called the **transitive closure** of R,
- If $R \subseteq X \times X$, then $R^* := R^+ \cup \{(x, x) \mid x \in X\}$ is called the **reflexive, transitive closure** of R.

Reflexive, transitive closure

Example:

$X = \{1, 2, 3, 4, 5\}$, $R = \{(1, 2), (2, 3), (3, 4), (2, 5)\}$. Then

$S = \{(1, 2), (2, 3), (3, 4), (2, 5), (1, 1), (2, 2), (3, 3), (4, 4), (5, 5)\}$ is the reflexive closure of R.

$R^1 = R = \{(1, 2), (2, 3), (3, 4), (2, 5)\}$,

$R^2 = \{(1, 3), (2, 4), (1, 5)\}$,

$R^3 = \{(1, 4)\}$,

$R^4 = R^5 = \ldots = \emptyset$.

$R^+ = \{(1, 2), (2, 3), (3, 4), (2, 5), (1, 3), (2, 4), (1, 5), (1, 4)\}$ is the transitive closure of R.

$R^* = \{(1, 2), (2, 3), (3, 4), (2, 5), (1, 3), (2, 4), (1, 5), (1, 4), (1, 1), (2, 2), (3, 3), (4, 4), (5, 5)\}$ is the reflexive, transitive closure of R.

Grammars

generated language

Definition

Let $G = (N, \Sigma, P, S)$ be a grammar. Then $L(G) := \{w \mid S \Rightarrow_G^* w, w \in \Sigma^*\}$ is called the **language generated by the grammar** G.

Words over Σ are called **terminal words.**

Observe, that $L(G)$ is the set of those sentential forms, that are terminal words. (These words are often called sentences.)

Grammars

generated language - examples

1st example

$G_1 = (N, \Sigma, P, S)$, where $N = \{S\}, \Sigma = \{a, b\}$ and $P = \{S \to aSb, S \to ab, S \to ba\}$.

Then $L(G_1) = \{a^n abb^n, a^n bab^n \mid n \geq 0\}$.

Explanation: Derivations in G_1 are special. The only option to generate a terminal word is applying the first rule n times ($n \geq 0$), then applying either rule 2 or rule 3 once.

This holds by the following inductive argument. Easy to see by induction, that all sentential forms contain at most one nonterminal symbol. Therefore, the first application of rule 2 or 3 stops the derivation.

It can be seen by induction, that the sentential form after n applications of the first rule is $a^n Sb^n$ implying the claim. (Induction step: $a^n Sb^n \Rightarrow a^n aSbb^n = a^{n+1} Sb^{n+1}$.)

Grammars

generated language - examples

2nd example

$G_2 = (N, \Sigma, P, S)$, where $N = \{S, A, B\}, \Sigma = \{a, b\}$ and $P = \{S \to ASB, S \to \varepsilon, AB \to BA, BA \to AB, A \to a, B \to b\}$.

Notation: Let $N_t(u)$ denote the number occurances of letter t in u.

Then $L(G_2) = \left\{u \in \{a, b\}^* \,\middle|\, N_a(u) = N_b(u)\right\}$, so G_2 generates exactly those words over $\{a, b\}$ containing an equal amount of a's and b's.

($\subseteq$:)

We can see by induction, that for all sentential forms α we have

$$N_a(\alpha) + N_A(\alpha) = N_b(\alpha) + N_B(\alpha).$$

Initially (i.e., for $\alpha = S$), equality holds. One can check, that none of the 6 rules can change the balance of the 2 sides.

Since all words of $L(G_2)$ are terminal words, we have $N_a(u) = N_b(u)$ for all words of $L(G_2)$.

Grammars

generated language - examples

($\supseteq$:)

For a terminal word u containing exactly n a's b's we give a derivation.

- apply the first rule n times giving the sentential form $A^n SB^n$. (One can check this by induction, as in example 1.)
- apply rule 2, so we have $A^n B^n$,
- using rules 3 and 4 any order of A's and B's can be generated, make the same order of A's and B's as it is in u,
- apply rules 5 and 6 to switch capital letters to small letters.

A derivation of *bbaaba* is the following.

$S \Rightarrow ASB \Rightarrow AASBB \Rightarrow AAASBBB \Rightarrow AAABBB \Rightarrow AABABB \Rightarrow ABAABB \Rightarrow BAAABB \Rightarrow BAABAB \Rightarrow BABAAB \Rightarrow BBAAAB \Rightarrow BBAABA \Rightarrow bBAABA \Rightarrow bbAABA \Rightarrow bbaABA \Rightarrow bbaaBA \Rightarrow bbaabA \Rightarrow bbaaba$.